

Module 03: Schedules & Serializability Foundations

Schedules & Serializability Foundations

Serial Schedules, Non-Serial Interleaving, and the Definition of Equivalence

1. Schedule Taxonomy & Classification

Execution Sequences of Interleaved Operations

Serial Schedule (Zero Interleaving)

Transactions execute strictly one after another.
Next transaction starts ONLY after current commits.
✓ 100% Consistent by definition! ✗ Poor throughput.

Non-Serial Schedule (Concurrent Interleaving)

Operations of multiple transactions are interleaved.
Maximizes CPU/Disk overlap and system responsiveness.
⚠ May lead to severe inconsistency if not serializable!

Permutations & Combinatorics Formula:

For n transactions: Total Serial Schedules = $n!$
For transactions with $n_1, n_2, \dots, n_k$ operations:
Total Non-Serial Schedules = $(n_1 + n_2 + \dots + n_k)! / (n_1! \cdot n_2! \cdot \dots \cdot n_k!)$

2. Concept of Serializability

The Benchmark for Concurrent Schedule Correctness

Formal Serializability Definition:

A non-serial schedule S is defined as Serializable if its final database state is EQUIVALENT to the outcome of some serial schedule S_{serial} of the same transactions.
Offers speed of interleaving with correctness of serial order!

Two Formal Types of Serializability:

- Conflict Serializability: Swapping non-conflicting ops.
Fast polynomial test $O(V + E)$ via Precedence Graph!
- View Serializability: Matching initial, updated & final reads.
NP-Complete problem! Handled via blind writes.

Mathematical Hierarchy:

Conflict Serializable $\subset$ View Serializable $\subset$ All Schedules
Every Conflict Serializable schedule is ALWAYS View Serializable.
View Serializable schedule without Blind Writes = Conflict Serializable!

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Definition of Schedule, Serial vs Non-Serial Schedules, Combinatorics of Schedules, Serializability as the Benchmark of Correctness, Conflict vs View Serializability Hierarchy.

1. What is a Schedule?

Jab multiple transactions system me ek saath run hoti hain, toh unke operations (read, write, commit, abort) jis chronological sequence (time order) me CPU par execute hote hain, use **Schedule** (ya History) kehte hain.

Formal Definition: A schedule \$\$\$ of n transactions $\{T_1, T_2, \dots, T_n\}$ is an ordering of the operations of the transactions such that for each transaction T_i , the operations of T_i in \$\$\$ appear in the same order as in T_i .

2. Classification of Schedules

2.1 Serial Schedule

Ek transaction ke saare operations ek saath execute hote hain. Doosra transaction tabhi shuru hota hai jab pehla **Commit** ho chuka ho.

Example: T_1 executes completely \$

$\rightarrow$ then T_2 executes completely (T_1

$\rightarrow T_2$).

Properties:

- Database hamesha **100% consistent** rehta hai (no concurrency hazards).
- CPU utilization bohot bekar hoti hai; response time bohot high hota hai (Zero concurrency).

2.2 Non-Serial (Interleaved) Schedule

Transactions ke operations ek doosre ke beech me mix (interleave) hokar run hote hain.

Properties:

- High system throughput aur fast response time.
- Lekin agar control na kiya jaye, toh inconsistent database ban sakta hai!

2.3 Combinatorics Formula (AKTU Objective & Gate Questions)

Agar relation me n transactions hain:

1. Total Number of Serial Schedules:

$$\text{Total Serial} = n!$$

2. Total Possible Concurrent Schedules:

Agar transaction T_1 me n_1 operations hain, T_2 me n_2 operations hain, ..., T_k me n_k operations hain:

$$\text{Total Schedules} =$$

$$\frac{(n_1 + n_2 + \dots + n_k)!}{n_1! \times n_2! \times \dots \times n_k!}$$

3. The Concept of Serializability

Golden Rule of Serializability:

Ek concurrent (non-serial) schedule SS tabhi **Correct** maana jaata hai jab uska final effect kisi ek **Serial Schedule** ke barabar ho!

Is property ko **Serializability** kehte hain.

Serial schedule benchmark hota hai.

Agar non-serial schedule serial schedule ke barabar outcome de raha hai, toh hume concurrency ki **high speed** bhi mil gayi aur serial execution ki **100% consistency** bhi mil gayi!

4. Types of Serializability

Serializability ko verify karne ke do theoretical approaches hain:

1. Conflict Serializability:

- Non-conflicting adjacent operations ko swap karke serial schedule me convert karna.
- Tested using **Precedence Graph (Cycle Detection)** in polynomial time $O(V + E)$.

2. View Serializability:

- Har transaction ke initial read, intermediate updated read, aur final write ko serial schedule se match karna.
- Testing view serializability is **NP-Complete!**

4.1 Master Inclusion Hierarchy

$$\mathbf{\text{Conflict Serializable} \subseteq \text{View Serializable} \subseteq \text{All Valid Schedules}}$$

Har Conflict Serializable schedule **by default View Serializable hota hai.**

Ek View Serializable schedule jo conflict serializable nahi hai, usme compulsory roop se **Blind Writes** ($W(A)$ without prior $R(A)$) present honge!

5. Architectural Diagram
